

Introduction to Access Control with Amazon Managed Streaming for Apache Kafka

SPL-BE-200-ANDES2-1 - Version 1.0.4

Lab overview

You are a data engineer at AnyCompany Online Shopping. AnyCompany Online Shopping sells a variety of consumer goods through e-commerce transactions in five categories: books, food, apparel, electronics, and home goods.

Your company wants a low latency solution in real-time to analyze web clickstream events from its online customers who interact with their retail website. Your executive team wants to see top sales by category updated in near real time. It wants a solution with no setup costs and without having to manage servers. The team decides to build its solution on Amazon Web Services (AWS) using Amazon Managed Streaming for Apache Kafka (Amazon MSK) and Amazon Managed Service for Apache Flink services.

You are responsible to build a real-time streaming analytics pipeline in Managed Apache Flink Studio to meet the requirements. Before you build the streaming pipeline, you want to understand AWS Identity and Access Management (IAM) access control for Amazon MSK. You can use this to handle both authentication and authorization for your MSK cluster. The Amazon MSK IAM access control method avoids the need to use one mechanism for authentication and another for authorization.

In this lab, you learn about the IAM method to authenticate and authorize users of an MSK cluster.

Objectives

By the end of this lab, you should be able to do the following:

- Publish to and consume from an MSK cluster using IAM authenticated broker Uniform Resource Locators (URLs) with a Java demo producer and Java demo consumer.
- Learn about the IAM method to authenticate and authorize users of an MSK cluster.

Start lab

1. To launch the lab, at the top of the page, choose **Start Lab**.
Caution: You must wait for the provisioned AWS services to be ready before you can continue.
2. To open the lab, choose **Open Console**.

You are automatically signed in to the AWS Management Console in a new web browser tab.

Warning: Do not change the **Region** unless instructed.

Lab environment

The following diagram shows the basic architecture of the lab environment:

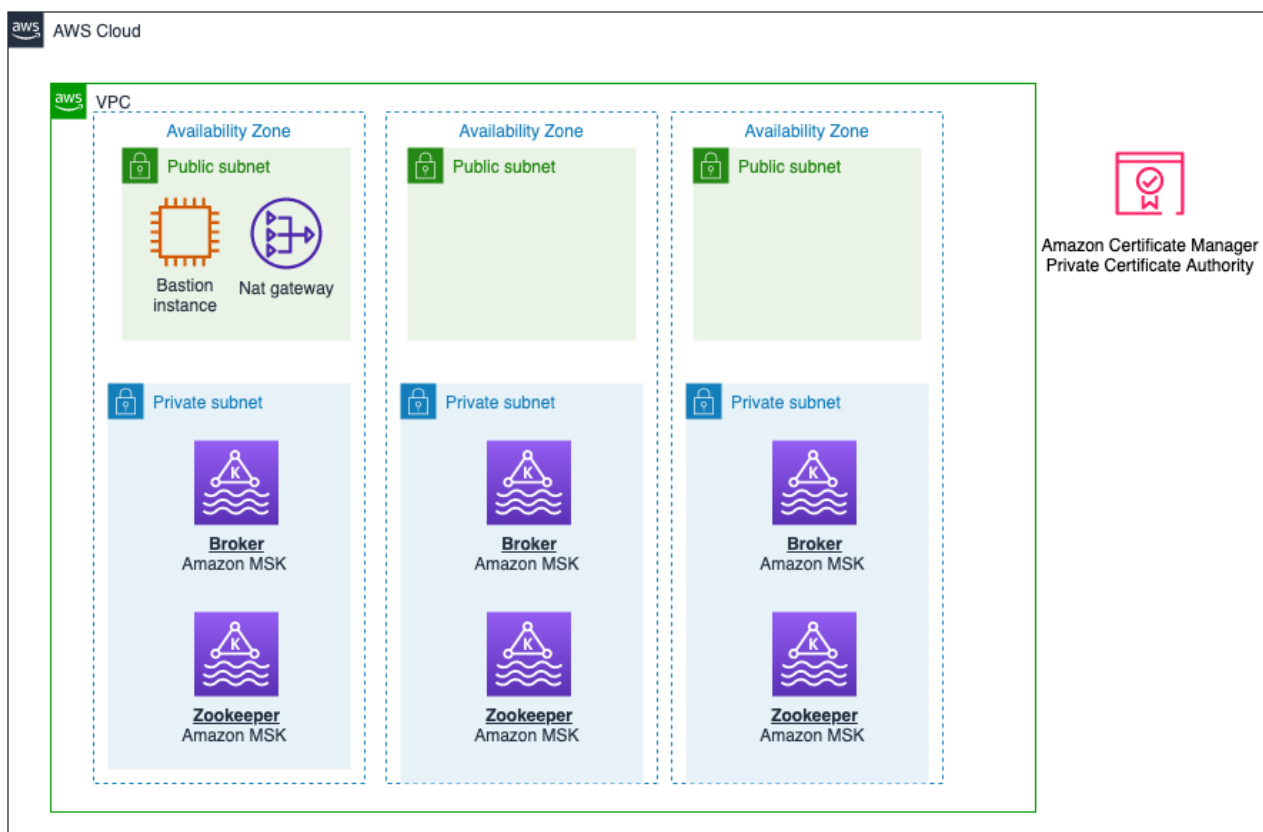


Image description: The preceding diagram depicts the lab environment which includes an Amazon MSK environment configured in a typical six-subnet (three public, three private) virtual private cloud (VPC). One public subnet contains a NAT gateway and a bastion host, which you use in the lab.

The provisioned MSK cluster has been configured for the IAM authentication method.

Task 1: Inspecting the MSK cluster

In this task, you navigate to the Amazon Elastic Compute Cloud (Amazon EC2) instance that serves as the Amazon MSK bastion host through a terminal session window. You run the command to display the output of a shell script that has been created to populate your environment with the various network addresses needed to access the MSK cluster.

Session Manager, a capability of AWS Systems Manager, provides secure instance management without the need to open inbound ports, maintain bastion hosts, or manage Secure Shell (SSH) keys.

Note: If the command shell is unresponsive when you type a command, to return to the shell prompt, press **Ctrl+C**.

3. To access Session Manager, in the list to the left of these instructions, copy the **CommandHostSessionManagementUrl** value.
4. To display the Amazon EC2 (Amazon MSK bastion host) terminal window, paste the URL into a new web browser tab and press **Enter**.
5. **Command:** To output the various network addresses needed to access the MSK cluster, run the following command:
`cat /opt/kafka_2.13-3.5.1/msk.env`

Expected output:

6. **** EXAMPLE OUTPUT ****
7. *****
- 8.

```

9. export PATH=/usr/local/sbin:/usr/local/bin:/usr/
   sbin:/usr/bin:/opt/kafka_2.13-3.5.1/bin:/opt/apache-
   maven-3.8.9/bin:/usr/local/bin/
10. export MSK_ARN=arn:aws:kafka:us-
   west-2:351259472416:cluster/MSK-Demo/
   cae08c93-2447-4b27-9c3f-e49f20b5e470-7
11. export MSK_BOOTSTRAP="b-1.msksdemo.q3xo2t.c7.kafka.us-
   west-2.amazonaws.com:9094,b-2.msksdemo.q3xo2t.c7.kafka
   .us-
   west-2.amazonaws.com:9094,b-3.msksdemo.q3xo2t.c7.kafka
   .us-west-2.amazonaws.com:9094"
12. export MSK_ZOOKEEPER="z-2.msksdemo.q3xo2t.c7.kafka.us-
   west-2.amazonaws.com:2181,z-3.msksdemo.q3xo2t.c7.kafka
   .us-
   west-2.amazonaws.com:2181,z-1.msksdemo.q3xo2t.c7.kafka
   .us-west-2.amazonaws.com:2181"
13. export
   MSKIAM_BOOTSTRAP="b-2.msksdemo.q3xo2t.c7.kafka.us-
   west-2.amazonaws.com:9098,b-3.msksdemo.q3xo2t.c7.kafka
   .us-
   west-2.amazonaws.com:9098,b-1.msksdemo.q3xo2t.c7.kafka
   .us-west-2.amazonaws.com:9098"

```

The output is a series of export statements that include the cluster Amazon Resource Names (ARNs), bootstrap brokers, and the Apache Zookeeper address for the cluster.

Note: The broker URLs in **MSK_BOOTSTRAP** uses TLS authenticated **port 9094** and the broker URLs in **MSKIAM_BOOTSTRAP** uses IAM authenticated **port 9098** for communication with the MSK cluster.

Task complete: You have successfully inspected the MSK cluster.

Task 2: Publish to and consume from an IAM authenticated MSK cluster

In this task, you publish and consume the messages from the topic using a console consumer. IAM access control for Amazon MSK handles both

authentication and authorization of producers and consumers that communicate with the cluster.

Caution: This lab has been pre-configured with these settings and no additional action is required. You create a topic and publish to the topic in Task 2.1.

Note: The following information explains the steps to enable IAM authentication in your own environment.

- As part of enabling IAM authentication from a bastion host, an IAM role needs to be attached to the bastion host. This role should include permissions for all MSK cluster actions.
- As part of enabling IAM authentication for the producers and consumers written in Java, the following lines of code needs to be added to the Kafka client configuration.

```
*****  
**** EXAMPLE ONLY, DO NOT COPY/PASTE ****  
*****
```

```
props.put("security.protocol", "SASL_SSL");  
props.put("sasl.mechanism", "AWS_MSK_IAM");  
props.put("sasl.jaas.config",  
"software.amazon.msk.auth.iam.IAMLoginModule required;");  
props.put("sasl.client.callback.handler.class",  
"software.amazon.msk.auth.iam.IAMClientCallbackHandler");
```

- The following lines of code needs to be added to the **<dependencies>** in the **pom.xml** file.

```
*****  
**** EXAMPLE ONLY, DO NOT COPY/PASTE ****  
*****
```

```
<dependency>  
  <groupId>software.amazon.msk</groupId>  
  <artifactId>aws-msk-iam-auth</artifactId>  
  <version>1.0.0</version>  
</dependency>
```

Learn more: Refer to *IAM access control* in the **Additional resources** section for more information.

Task 2.1: Create a topic, compile a Java producer/consumer and publish to an IAM authenticated MSK cluster

Create and list a topic.

6. **Command:** To load the environment variable that references the MSK cluster into your shell environment, run the following command:

```
source /opt/kafka_2.13-3.5.1/msk.env
```

Expected output:

None, unless there is an error.

7. **Command:** To create a topic named **ExampleTopic**, run the following command:

```
kafka-topics.sh --create --topic ExampleTopic --  
partitions 5 --replication-factor 3 --bootstrap-  
server $MSK_BOOTSTRAP --command-config /opt/  
client.properties
```

Expected output:

None, unless there is an error.

8. **Command:** To list the topics on the cluster to verify successful creation, run the following command:

```
kafka-topics.sh --list --bootstrap-server  
$MSK_BOOTSTRAP --command-config /opt/  
client.properties
```

Expected output:

```
*****
```

9. **** EXAMPLE OUTPUT ****

- 10.*****

- 11.

```
12.ExampleTopic
13.__amazon_msk_canary
14.__consumer_offsets
```

You see a list of topics, which includes a couple that were created at cluster creation, and the topic **ExampleTopic** that you created earlier.

Task 2.2: Compile Java producer and consumer

Communication with an IAM authenticated broker would require a Java based Kafka client. The source code for a Java Kafka producer and consumer has been loaded onto the bastion host. It is located at */opt/msk-java*.

9. **Command:** To navigate to the */opt/msk-java* directory, run the following command:
- ```
cd /opt/msk-java
```

**Expected output:**

*None, unless there is an error.*

Within this directory is a Maven project that has already been set up.

10. **Command:** To compile the project, run the following command:
- ```
mvn package
```

Expected output: Output text has been truncated.

```
*****
```

```
11.**** EXAMPLE OUTPUT ****
12.*****
13.
14....
15.[ INFO]
```

```
-----
-----
```

```
16.[ INFO] BUILD SUCCESS
```

17.[INFO]

18.[INFO] Total time: 16.118 s

19.[INFO] Finished at: 2025-07-22T15:55:27Z

20.[INFO]

When this task is complete, you see **BUILD SUCCESS** and are returned back to the bash shell.

Task 2.3: Publish to the topic

11. **Command:** To publish to **ExampleTopic** with the Java program that you compiled in the previous step using IAM authenticated broker URLs, run the following command:

```
java -cp target/msk-auth-demo-1.0-SNAPSHOT.jar  
com.amazonaws.examples.DemoProducer $MSKIAM_BOOTSTRAP  
ExampleTopic
```

Expected output:

12.**** EXAMPLE OUTPUT ****

13.*****

14.

15.[main] INFO

org.apache.kafka.clients.producer.ProducerConfig -
ProducerConfig values:

16.acks = all

17.batch.size = 16384

18.bootstrap.servers =

[b-1.msksdemo.7zfurd.c7.kafka.us-
east-1.amazonaws.com:9098,
b-2.msksdemo.7zfurd.c7.kafka.us-
east-1.amazonaws.com:9098,

```

b-3.mskdemo.7zfurd.c7.kafka.us-
east-1.amazonaws.com:9098]
19.         buffer.memory = 33554432
20.         client.dns.lookup = default
21.         client.id =
22.         compression.type = none
23.         connections.max.idle.ms = 540000
24.         delivery.timeout.ms = 120000
25.         enable.idempotence = false
26.         interceptor.classes = []
27.         key.serializer = class
org.apache.kafka.common.serialization.StringSerialize
r
28.         linger.ms = 1
29.         max.block.ms = 60000
30.         max.in.flight.requests.per.connection = 5
31.         max.request.size = 1048576
32.         metadata.max.age.ms = 300000
33.         metric.reporters = []
34.         metrics.num.samples = 2
35.         metrics.recording.level = INFO
36.         metrics.sample.window.ms = 30000
37.         partitioner.class = class
org.apache.kafka.clients.producer.internals.DefaultPa
rtitioner
38.         receive.buffer.bytes = 32768
39.         reconnect.backoff.max.ms = 1000
40.         reconnect.backoff.ms = 50
41.         request.timeout.ms = 30000
42.         retries = 0
43.         retry.backoff.ms = 100
44.         sasl.client.callback.handler.class = class
software.amazon.msk.auth.iam.IAMClientCallbackHandler
45.         sasl.jaas.config = [hidden]
46.         sasl.kerberos.kinit.cmd = /usr/bin/kinit
47.         sasl.kerberos.min.time.before.relogin = 60000
48.         sasl.kerberos.service.name = null
49.         sasl.kerberos.ticket.renew.jitter = 0.05
50.         sasl.kerberos.ticket.renew.window.factor =
0.8
51.         sasl.login.callback.handler.class = null
52.         sasl.login.class = null

```

```
53.         sasl.login.refresh.buffer.seconds = 300
54.         sasl.login.refresh.min.period.seconds = 60
55.         sasl.login.refresh.window.factor = 0.8
56.         sasl.login.refresh.window.jitter = 0.05
57.         sasl.mechanism = AWS_MSK_IAM
58.         security.protocol = SASL_SSL
59.         send.buffer.bytes = 131072
60.         ssl.cipher.suites = null
61.         ssl.enabled.protocols = [TLSv1.2, TLSv1.1,
    TLSv1]
62.         ssl.endpoint.identification.algorithm = https
63.         ssl.key.password = null
64.         ssl.keymanager.algorithm = SunX509
65.         ssl.keystore.location = null
66.         ssl.keystore.password = null
67.         ssl.keystore.type = JKS
68.         ssl.protocol = TLS
69.         ssl.provider = null
70.         ssl.secure.random.implementation = null
71.         ssl.trustmanager.algorithm = PKIX
72.         ssl.truststore.location = null
73.         ssl.truststore.password = null
74.         ssl.truststore.type = JKS
75.         transaction.timeout.ms = 60000
76.         transactional.id = null
77.         value.serializer = class
    org.apache.kafka.common.serialization.StringSerialize
    r
78.
79.[main] INFO
    org.apache.kafka.common.security.authenticator.Abstra
    ctLogin - Successfully logged in.
80.[main] WARN
    org.apache.kafka.clients.producer.ProducerConfig -
    The configuration 'min.insync.replicas' was supplied
    but isn't a known config.
81.[main] INFO
    org.apache.kafka.common.utils.AppInfoParser - Kafka
    version: 2.2.1
82.[main] INFO
    org.apache.kafka.common.utils.AppInfoParser - Kafka
    commitId: 55783d3133a5a49a
```

```
83.>[kafka-producer-network-thread | producer-1] INFO
    org.apache.kafka.clients.Metadata - Cluster ID:
    lymb5mcpTfid-G4Hg5CR7Q
```

Add a message to the queue.

84. From the blinking cursor, enter a sample message
Hello World!, and at the > prompt press **ENTER**.

Expected output:

```
85.**** EXAMPLE OUTPUT ****
```

```
86.*****
```

```
87.
```

```
88.Hello World!
```

```
89.sent message to topic:ExampleTopic partition:1
    offset:0
```

```
90.>
```

Keep the terminal window open as you proceed to the following sub-task.

Note: The broker URLs in *MSKIAM_BOOTSTRAP* uses IAM authenticated *port 9098* to publish to and consume from a Java client. You have successfully created a topic on the cluster and published a message to the topic using a Java demo producer.

Task 2.4: Consume from an IAM authenticated MSK cluster

In this sub-task, you consume the messages from the created topic.

Setup a new terminal window in a new browser tab.

13. To access Session Manager, in the list to the left of these instructions, copy the **CommandHostSessionManagementUrl** value.

14. To display the Amazon EC2 (Amazon MSK bastion host) terminal window, paste the URL into a new web browser tab and press **Enter**.

15. **Command:** To load the environment variable that references the MSK cluster into your shell environment, run the following command:

```
source /opt/kafka_2.13-3.5.1/msk.env
```

Expected output:

None, unless there is an error.

Caution: Continue the rest of the sub-task in the same session window.

Note: The source code for a Java Kafka producer and consumer has been loaded onto the lab bastion host. It is located at */opt/msk-java*.

16. **Command:** To navigate to the */opt/msk-java* directory, run the following command:

```
cd /opt/msk-java
```

Expected output:

None, unless there is an error.

17. **Command:** To consume the messages using a Java client with IAM authenticated broker URLs, run the following command:

```
java -cp target/msk-auth-demo-1.0-SNAPSHOT.jar  
com.amazonaws.examples.DemoConsumer $MSKIAM_BOOTSTRAP  
ExampleTopic
```

Expected output:

18. **** EXAMPLE OUTPUT ****

19. *****

20.

21. [main] INFO

```
org.apache.kafka.clients.consumer.ConsumerConfig -  
ConsumerConfig values:
```

22. auto.commit.interval.ms = 5000

23. auto.offset.reset = earliest

24. bootstrap.servers =

```
[b-1.msksdemo.7zfurd.c7.kafka.us-
```

```

    east-1.amazonaws.com:9098,
    b-2.msksdemo.7zfurd.c7.kafka.us-
    east-1.amazonaws.com:9098,
    b-3.msksdemo.7zfurd.c7.kafka.us-
    east-1.amazonaws.com:9098]
25.         check.crcs = true
26.         client.dns.lookup = default
27.         client.id =
28.         connections.max.idle.ms = 540000
29.         default.api.timeout.ms = 60000
30.         enable.auto.commit = false
31.         exclude.internal.topics = true
32.         fetch.max.bytes = 52428800
33.         fetch.max.wait.ms = 500
34.         fetch.min.bytes = 1
35.         group.id = foobar
36.         heartbeat.interval.ms = 3000
37.         interceptor.classes = []
38.         internal.leave.group.on.close = true
39.         isolation.level = read_uncommitted
40.         key.deserializer = class
    org.apache.kafka.common.serialization.StringDeseriali
zer
41.         max.partition.fetch.bytes = 1048576
42.         max.poll.interval.ms = 300000
43.         max.poll.records = 500
44.         metadata.max.age.ms = 300000
45.         metric.reporters = []
46.         metrics.num.samples = 2
47.         metrics.recording.level = INFO
48.         metrics.sample.window.ms = 30000
49.         partition.assignment.strategy = [class
    org.apache.kafka.clients.consumer.RangeAssignor]
50.         receive.buffer.bytes = 65536
51.         reconnect.backoff.max.ms = 1000
52.         reconnect.backoff.ms = 50
53.         request.timeout.ms = 30000
54.         retry.backoff.ms = 100
55.         sasl.client.callback.handler.class = class
    software.amazon.msk.auth.iam.IAMClientCallbackHandler
56.         sasl.jaas.config = [hidden]
57.         sasl.kerberos.kinit.cmd = /usr/bin/kinit

```

```

58.         sasl.kerberos.min.time.before.relogin = 60000
59.         sasl.kerberos.service.name = null
60.         sasl.kerberos.ticket.renew.jitter = 0.05
61.         sasl.kerberos.ticket.renew.window.factor =
0.8
62.         sasl.login.callback.handler.class = null
63.         sasl.login.class = null
64.         sasl.login.refresh.buffer.seconds = 300
65.         sasl.login.refresh.min.period.seconds = 60
66.         sasl.login.refresh.window.factor = 0.8
67.         sasl.login.refresh.window.jitter = 0.05
68.         sasl.mechanism = AWS_MSK_IAM
69.         security.protocol = SASL_SSL
70.         send.buffer.bytes = 131072
71.         session.timeout.ms = 10000
72.         ssl.cipher.suites = null
73.         ssl.enabled.protocols = [TLSv1.2, TLSv1.1,
TLSv1]
74.         ssl.endpoint.identification.algorithm = https
75.         ssl.key.password = null
76.         ssl.keymanager.algorithm = SunX509
77.         ssl.keystore.location = null
78.         ssl.keystore.password = null
79.         ssl.keystore.type = JKS
80.         ssl.protocol = TLS
81.         ssl.provider = null
82.         ssl.secure.random.implementation = null
83.         ssl.trustmanager.algorithm = PKIX
84.         ssl.truststore.location = null
85.         ssl.truststore.password = null
86.         ssl.truststore.type = JKS
87.         value.deserializer = class
org.apache.kafka.common.serialization.StringDeseriali
zer
88.
89.[main] INFO
org.apache.kafka.common.security.authenticator.Abstra
ctLogin - Successfully logged in.
90.[main] WARN
org.apache.kafka.clients.consumer.ConsumerConfig -
The configuration 'batch.size' was supplied but isn't
a known config.

```

91.[main] WARN
org.apache.kafka.clients.consumer.ConsumerConfig -
The configuration 'acks' was supplied but isn't a
known config.

92.[main] WARN
org.apache.kafka.clients.consumer.ConsumerConfig -
The configuration 'buffer.memory' was supplied but
isn't a known config.

93.[main] WARN
org.apache.kafka.clients.consumer.ConsumerConfig -
The configuration 'retries' was supplied but isn't a
known config.

94.[main] WARN
org.apache.kafka.clients.consumer.ConsumerConfig -
The configuration 'linger.ms' was supplied but isn't
a known config.

95.[main] INFO
org.apache.kafka.common.utils.AppInfoParser - Kafka
version: 2.2.1

96.[main] INFO
org.apache.kafka.common.utils.AppInfoParser - Kafka
commitId: 55783d3133a5a49a

97.[main] INFO
org.apache.kafka.clients.consumer.KafkaConsumer -
[Consumer clientId=consumer-1, groupId=foobar]
Subscribed to topic(s): ExampleTopic

98.[main] INFO org.apache.kafka.clients.Metadata -
Cluster ID: lycb5mcpTfid-G4Hg5CR7Q

99.[main] INFO
org.apache.kafka.clients.consumer.internals.AbstractC
oordinator - [Consumer clientId=consumer-1,
groupId=foobar] Discovered group coordinator
b-2.mskdemo.7zfurd.c7.kafka.us-
east-1.amazonaws.com:9098(id: 2147483645 rack: null)

100.[main] INFO
org.apache.kafka.clients.consumer.internals.ConsumerC
oordinator - [Consumer clientId=consumer-1,
groupId=foobar] Revoking previously assigned
partitions []

101.[main] INFO
org.apache.kafka.clients.consumer.internals.AbstractC

```
oordinator - [Consumer clientId=consumer-1,
groupId=foobar] (Re-)joining group
102.[main] INFO
org.apache.kafka.clients.consumer.internals.AbstractC
oordinator - [Consumer clientId=consumer-1,
groupId=foobar] (Re-)joining group
103.[main] INFO
org.apache.kafka.clients.consumer.internals.AbstractC
oordinator - [Consumer clientId=consumer-1,
groupId=foobar] Successfully joined group with
generation 1
104.[main] INFO
org.apache.kafka.clients.consumer.internals.ConsumerC
oordinator - [Consumer clientId=consumer-1,
groupId=foobar] Setting newly assigned partitions:
ExampleTopic-0, ExampleTopic-2, ExampleTopic-1,
ExampleTopic-4, ExampleTopic-3
105.[main] INFO
org.apache.kafka.clients.consumer.internals.Fetcher -
[Consumer clientId=consumer-1, groupId=foobar]
Resetting offset for partition ExampleTopic-0 to
offset 0.
106.[main] INFO
org.apache.kafka.clients.consumer.internals.Fetcher -
[Consumer clientId=consumer-1, groupId=foobar]
Resetting offset for partition ExampleTopic-3 to
offset 0.
107.[main] INFO
org.apache.kafka.clients.consumer.internals.Fetcher -
[Consumer clientId=consumer-1, groupId=foobar]
Resetting offset for partition ExampleTopic-1 to
offset 0.
108.[main] INFO
org.apache.kafka.clients.consumer.internals.Fetcher -
[Consumer clientId=consumer-1, groupId=foobar]
Resetting offset for partition ExampleTopic-4 to
offset 0.
109.[main] INFO
org.apache.kafka.clients.consumer.internals.Fetcher -
[Consumer clientId=consumer-1, groupId=foobar]
Resetting offset for partition ExampleTopic-2 to
offset 0.
```

110.Hello World!

It takes a few seconds to see the first sample message *Hello World!*.

111.Navigate to the **Java producer session window** and enter further sample messages at the > prompt.

112.Enter

It is a great day. and press **Enter**.

Expected output:

113.**** EXAMPLE OUTPUT ****

114.*****

115.

116.sent message to topic:ExampleTopic partition:4
offset:0

117.>

118.Enter

I got this. and press **Enter**.

Expected output:

119.**** EXAMPLE OUTPUT ****

120.*****

121.

122.sent message to topic:ExampleTopic partition:4
offset:1

123.>

124.Switch to the **Java consumer session window**.

Expected output:

125.**** EXAMPLE OUTPUT ****

126.*****

127.

128.It is a great day.
129.I got this.

You can see the new messages consumed in the Java consumer session window.

Note: You can close out of both the producer and consumer session windows by choosing **Terminate** in the top right corner of the session window.

Task complete: You have successfully published and consumed the messages from the topic using a console consumer.

Conclusion

You have successfully done the following:

- Published to and consumed from an MSK cluster using IAM authenticated broker URLs with a Java demo producer and Java demo consumer.
- Learned about the IAM method to authenticate and authorize users of an MSK cluster.

End lab

Follow these steps to close the console and end your lab.

22. Return to the **AWS Management Console**.
23. At the upper-right corner of the page, choose **AWS Labs User**, and then choose **Sign out**.
24. Choose **End Lab** and then confirm that you want to end your lab.

Additional resources

- [IAM access control](#)